Response Propensity Estimation for Combining Probability and Non-Probability Samples with LBBNN

Aliaksandr Hubin & Lars Skaaret-Lund (in collaboration with Vladislav Beresovsky, Julie Gershunskaya

2026-10-22

Contents

1 Executive Summary **2**

2 Theoretical Foundations **3**

2.1 The Challenge of Non-Probability Convenience Data . . . 3

2.2 The Five Estimation Formulations (Beresovsky, Gershunskaya & Savitsky, 2026) . . . 3

2.2.1 The Fundamental Identification Problem & The Conditioning Trick . . . 3

2.2.2 Unlinked Stacked Sample (`STACK`) . . . 4

2.2.3 Deterministically Linked Union (`LINK1`) . . . 4

2.2.4 Deterministically Linked Stacked Sample (`LINK2`) . . . 4

2.2.5 Missing Information Principle (`MIP`) . . . 5

2.2.6 Pseudo-Log-Likelihood (`PSEUDO`, Chen, Li & Wu 2020) . . . 5

2.3 Self-Normalizing Hájek Estimator . . . 6

2.4 Variational ELBO Calibration & Protection Against Cold Posteriors . . . 6

3 Package Setup & Environment Verification **6**

4 Simulation Study 0: Exact Linear Baseline (Beresovsky et al., 2026) **7**

4.1 High Common Support Benchmark . . . 10

4.2 Low Common Support Benchmark . . . 11

5 Simulation Study 1: High Common Support with Linear Propensity & Noise Covariates **11**

5.1 Fitting Propensity Models (LBBNN vs All 5 GLM Baselines) . . . 13

5.2 Sparse Variable Selection & Active Graph Pruning . . . 14

5.3 Finite Population Mean Recovery . . . 15

6 Simulation Study 2: High Common Support with Non-Linear Propensity & Interaction 16
6.1 Fitting Propensity Models under High Support Non-Linearity 17
6.2 Variable Selection under Non-Linear Interaction 17
6.3 Finite Population Mean Recovery 18
6.3.1 Pedagogical Takeaway 20

7 Simulation Study 3: Low Common Support with Non-Linear Propensity & Interaction 20
7.1 Model Fitting under Low Common Support 21
7.2 Performance Comparison: Kish's ESS & The Breakdown of Pseudo-Likelihood 21
7.3 Propensity Recovery Diagnostic: 2x2 Comparison Grid 23

8 Synthesis & Comparative Discussion Across Simulation Regimes 24
8.1 When the True Propensity is Linear (Studies 0 & 1) 26
8.2 When the Propensity is Non-Linear with Interactions under High Common Support (Study 2) 26
8.3 When Non-Linearity Meets Low Common Support (Study 3) 26

9 Collaborator API Reference 26

10 References 27

11 Session Information 27

1 Executive Summary

In federal and official statistics, the combination of a large **non-probability ("convenience") sample** (such as commercial web panels or administrative filings) with a smaller, high-quality **probability reference survey** offers substantial promise for reducing survey costs and respondent burden while expanding data coverage. However, unweighted convenience data suffer from severe **selection bias** (Meng, 2018).

This report implements and benchmarks the **Latent Binary Bayesian Neural Network (`LBBNN`)** framework for estimating unknown convenience participation propensities $\pi_{ci} = \mathbb{P}(I_{ci} = 1 \mid \mathbf{x}_i)$ under five composite likelihood and pseudo-likelihood formulations:

1. `STACK`: Composite Bernoulli likelihood on the unlinked stacked sample $S = S_c \oplus S_r$ ($\pi_{zi} = \frac{\pi_{ci}}{\pi_{ci}+\pi_{ri}}$).
2. `LINK1`: Deterministically linked union sample $S = S_c \cup (S_r \setminus S_c)$ ($\pi_{zi} = \frac{\pi_{ci}}{\pi_{ci}+\pi_{ri}-\pi_{ci}\pi_{ri}}$).
3. `LINK2`: Deterministically linked stacked sample $S = S_c \oplus S_r$ ($\pi_{zi} = \frac{\pi_{ci}(1+\pi_{ri})}{\pi_{ci}+\pi_{ri}}$).
4. `MIP` **(Missing Information Principle)**: Probabilistically linked stacked sample with linkage informativeness parameter $p_{\text{link}} \in [0, 1]$.
5. `PSEUDO`: Pseudo-log-likelihood approach of Chen, Li & Wu (2020) utilizing survey design weights $w_{ri} = 1/\pi_{ri}$ on S_r.

Through automated **variational spike-and-slab edge selection**, `LBBNN` simultaneously: - Captures complex **non-linear propensity curves and multi-way interactions** without manual polynomial expansions. - Prunes **uninformative noise covariates** from the network architecture. - Prevents the catastrophic **variance and weight explosion** that afflicts classical pseudo-likelihood under low common support.

2 Theoretical Foundations

2.1 The Challenge of Non-Probability Convenience Data

Let $U = \{1, \ldots, N\}$ denote a finite target population. Each unit $i \in U$ is associated with: - A vector of auxiliary covariates $\mathbf{x}_i \in \mathbb{R}^p$. - An outcome of interest $y_i \in \mathbb{R}$. - An unobserved convenience sample participation indicator $I_{ci} \in \{0, 1\}$, with unknown participation propensity $\pi_{ci} = \mathbb{P}(I_{ci} = 1 \mid \mathbf{x}_i) > 0$. - A probability survey inclusion indicator $I_{ri} \in \{0, 1\}$, with known design selection probability $\pi_{ri} = \mathbb{P}(I_{ri} = 1 \mid \mathbf{x}_i) > 0$.

If units with particular values of $\mathbf{x}_i$ or y_i systematically self-select into the convenience dataset, the naive sample mean is severely biased:

$$\text{Bias}(\bar{y}_c) = \frac{\text{Cov}_U(I_c, Y)}{\bar{\pi}_c}$$

As Meng (2018) proved, this selection bias scales with $\sqrt{N}$ rather than sample size n_c: even a massive convenience dataset comprising 80% or 90% of the population can produce wildly misleading population estimates if $\text{Corr}(I_c, Y) \neq 0$.

2.2 The Five Estimation Formulations (Beresovsky, Gershunskaya & Savitsky, 2026)

2.2.1 The Fundamental Identification Problem & The Conditioning Trick

In standard supervised machine learning, fitting a binary classification model $\mathbb{P}(I_{ci} = 1 \mid \mathbf{x}_i)$ requires a training sample of both positive ($I_{ci} = 1$) and negative ($I_{ci} = 0$) units drawn from the population. In the non-probability survey setting, however, we encounter a **fundamental identification barrier**: 1. We observe auxiliary covariates $\mathbf{x}_i$ only for units that participated in the convenience sample $S_c = \{i \in U : I_{ci} = 1\}$ and units selected into the reference probability survey $S_r = \{i \in U : I_{ri} = 1\}$. 2. For all non-sampled population units ($i \notin S_c \cup S_r$), the covariates $\mathbf{x}_i$ are entirely unobserved.

Consequently, one cannot directly fit a standard binary regression for I_{ci} over U. Beresovsky, Gershunskaya & Savitsky (2026) resolve this challenge through a **conditioning trick**: by combining S_c and S_r into a combined sample S and conditioning on the event that a unit was observed in S, the known reference design probability π_{ri} acts as a **structural offset** that anchors the neural network's unconstrained logits $\eta_\theta(\mathbf{x}_i)$ to the true population scale, allowing exact recovery of $\pi_{ci}(\mathbf{x}_i; \theta) = \sigma(\eta_\theta(\mathbf{x}_i))$.

Table 1: Overview of the Five Sample Combination Mechanisms
(where $D_i = \mathbb{I}(i \in S_c)$ denotes sample origin)

Estimator	Sample Domain	Target Indicator & Linkage	Composite Probability / Mechanism
STACK	$S_c \oplus S_r$	Unlinked: $I_{zi} = D_i$	$\pi_{zi} = \frac{\pi_{ci}}{\pi_{ci}+\pi_{ri}}$
LINK1	$S_c \cup (S_r \setminus S_c)$	Dedup. Union: $I_{zi} = I_{ci}$ ($S_r \cap S_c$ dropped)	$\pi_{zi} = \frac{\pi_{ci}}{\pi_{ci}+\pi_{ri}-\pi_{ci}\pi_{ri}}$
LINK2	$S_c \oplus S_r$	Linked Stacked: $I_{zi} = D_i + (1 - D_i)I_{ci}$	$\pi_{zi} = \frac{\pi_{ci}(1+\pi_{ri})}{\pi_{ci}+\pi_{ri}}$
MIP	$S_c \oplus S_r$	Prob. Linked ($p_{\text{link}} \in [0, 1]$): $\tilde{I}_{zi} = D_i + (1 - D_i)\tilde{p}_i$	$\pi_{zi} = \frac{\pi_{ci}(1+\pi_{ri})}{\pi_{ci}+\pi_{ri}}$ with $\tilde{p}_i = p_{\text{link}} I_{ci} + (1 - p_{\text{link}})\hat{\pi}_{ci}$
PSEUDO	$S_c \oplus S_r$	Unlinked Weighted: D_i with $w_{ri} = \pi_{ri}^{-1}$ on S_r	$\pi_{ci} = \sigma(\eta_\theta(\mathbf{x}_i))$ via weighted softplus loss

2.2.2 Unlinked Stacked Sample (`STACK`)

When records between the convenience and reference files cannot be matched, we form the stacked multiset $S = S_c \oplus S_r$ of size $n = n_c + n_r$. Any population unit $i \in S_c \cap S_r$ that happens to be selected into both files appears **twice** in S: once in the S_c block with origin indicator $D_i = 1$, and once in the S_r block with $D_i = 0$.

Defining the target response indicator as $I_{zi} = D_i$, we evaluate the conditional probability that a record with covariate profile $\mathbf{x}_i$ observed in $S_c \oplus S_r$ originated from S_c using Bayes' rule:

$$\pi_{zi}^{\text{STACK}} = \mathbb{P}(D_i = 1 \mid \mathbf{x}_i, i \in S_c \oplus S_r) = \frac{\mathbb{E}[I_{ci} \mid \mathbf{x}_i]}{\mathbb{E}[I_{ci} + I_{ri} \mid \mathbf{x}_i]} = \frac{\pi_{ci}}{\pi_{ci} + \pi_{ri}}$$

Writing the convenience propensity as $\pi_{ci} = \sigma(\eta_\theta(\mathbf{x}_i))$, the log-odds of π_{zi}^{STACK} satisfy:

$$\log\left(\frac{\pi_{zi}^{\text{STACK}}}{1 - \pi_{zi}^{\text{STACK}}}\right) = \log \pi_{ci} - \log \pi_{ri} = \eta_\theta(\mathbf{x}_i) - \log(1 + e^{\eta_\theta(\mathbf{x}_i)}) - \log \pi_{ri}$$

In typical survey settings where $\pi_{ci} \ll 1$, this behaves as an unconstrained binary classification model where $-\log \pi_{ri}$ acts as a known structural offset. Training minimizes the standard Bernoulli negative log-likelihood:

$$\mathcal{L}_{\text{STACK}}(\theta) = -\sum_{i \in S_c \oplus S_r} \left[I_{zi} \log(\pi_{zi}^{\text{STACK}}) + (1 - I_{zi}) \log(1 - \pi_{zi}^{\text{STACK}}) \right]$$

2.2.3 Deterministically Linked Union (`LINK1`)

When unique identifiers (such as Social Security or Employer Identification Numbers) are present, overlapping units $i \in S_r \cap S_c$ can be deterministically matched and removed from the reference survey, yielding the deduplicated union $S = S_c \cup (S_r \setminus S_c)$.

Assuming conditional independence $\mathbb{P}(I_{ci} = 1, I_{ri} = 1 \mid \mathbf{x}_i) = \pi_{ci}\pi_{ri}$ (which holds whenever the reference sampling mechanism is non-informative given $\mathbf{x}_i$), the marginal probability of population unit i entering the union is $\pi_{ci} + \pi_{ri} - \pi_{ci}\pi_{ri}$. Conditioning on membership in the union, the probability that unit i belongs to the convenience sample S_c ($I_{zi} = 1$) rather than the non-overlapping reference sample $S_r \setminus S_c$ ($I_{zi} = 0$) is:

$$\pi_{zi}^{\text{LINK1}} = \frac{\pi_{ci}}{\pi_{ci} + \pi_{ri} - \pi_{ci}\pi_{ri}}$$

$$\mathcal{L}_{\text{LINK1}}(\theta) = -\sum_{i \in S_c \cup (S_r \setminus S_c)} \left[I_{zi} \log(\pi_{zi}^{\text{LINK1}}) + (1 - I_{zi}) \log(1 - \pi_{zi}^{\text{LINK1}}) \right]$$

2.2.4 Deterministically Linked Stacked Sample (`LINK2`)

`LINK2` improves upon both `STACK` and `LINK1` by retaining the full sample size $n = n_c + n_r$ of the stacked multiset $S = S_c \oplus S_r$ without discarding observations, while exploiting deterministic linkage to **re-label** overlapping records within the S_r block.

Let $D_i \in \{0, 1\}$ denote whether record i belongs to the S_c file ($D_i = 1$) or the S_r file ($D_i = 0$). In `LINK2`, the target indicator is defined as:

$$I_{zi} = D_i + (1 - D_i) I_{ci}$$

Under this definition: - Every record in S_c receives $I_{zi} = 1$. - Any record in S_r that also participated in S_c ($I_{ci} = 1$) is flagged and receives $I_{zi} = 1$. - Only records in S_r that did *not* participate in S_c ($I_{ci} = 0$) receive $I_{zi} = 0$.

Across the two stacked files, the expected number of records at $\mathbf{x}_i$ assigned $I_{zi} = 1$ is $\mathbb{E}[I_{ci} + I_{ri}I_{ci} \mid \mathbf{x}_i] = \pi_{ci} + \pi_{ci}\pi_{ri} = \pi_{ci}(1 + \pi_{ri})$, while the total expected number of records is $\mathbb{E}[I_{ci} + I_{ri} \mid \mathbf{x}_i] = \pi_{ci} + \pi_{ri}$. This yields the composite conditional probability:

$$\pi_{zi}^{\text{LINK2}} = \frac{\pi_{ci}(1 + \pi_{ri})}{\pi_{ci} + \pi_{ri}}$$

$$\mathcal{L}_{\text{LINK2}}(\theta) = -\sum_{i \in S_c \oplus S_r} \left[I_{zi} \log(\pi_{zi}^{\text{LINK2}}) + (1 - I_{zi}) \log(1 - \pi_{zi}^{\text{LINK2}}) \right]$$

2.2.5 Missing Information Principle (`MIP`)

In administrative and survey practice, exact deterministic matching is rarely available; linkages are typically probabilistic (e.g., via Fellegi-Sunter record linkage or `fastLink`), yielding an estimated linkage probability $p_{\text{link}} \in [0, 1]$.

Under Louis's (1982) Missing Information Principle: - If linkage were perfectly deterministic ($p_{\text{link}} = 1$), $I_{ci} \in \{0, 1\}$ would be fully observed for all reference units $i \in S_r$, allowing direct evaluation of the complete-data `LINK2` score with indicator $I_{zi}^{\text{LINK2}} = D_i + (1 - D_i)I_{ci}$. - When linkage is imperfect ($p_{\text{link}} \in [0, 1]$), the true indicator I_{ci} on S_r is partially latent. Analogous to the Expectation step of an EM algorithm, we project the complete-data score onto the observed data by replacing I_{ci} with its conditional expectation:

$$\tilde{p}_i = p_{\text{link}} I_{ci} + (1 - p_{\text{link}}) \text{sg}\big(\pi_{ci}(\mathbf{x}_i; \theta)\big), \qquad \tilde{I}_{zi}^{\text{MIP}} = D_i + (1 - D_i)\tilde{p}_i$$

where $\text{sg}(\cdot)$ denotes the stop-gradient (`$detach()`) operator. This ensures that $\tilde{p}_i$ acts as a fixed target during backpropagation while gradients flow through the composite model probability $\pi_{zi} = \frac{\pi_{ci}(1+\pi_{ri})}{\pi_{ci}+\pi_{ri}}$:

$$\mathcal{L}_{\text{MIP}}(\theta) = -\sum_{i \in S_c \oplus S_r} \left[\tilde{I}_{zi}^{\text{MIP}} \log(\pi_{zi}) + (1 - \tilde{I}_{zi}^{\text{MIP}}) \log(1 - \pi_{zi}) \right]$$

Importantly, `MIP` continuously interpolates between the unlinked and deterministically linked extremes: - When $p_{\text{link}} = 1$, $\tilde{I}_{zi}^{\text{MIP}} = I_{zi}^{\text{LINK2}}$, reproducing `LINK2` exactly. - When $p_{\text{link}} = 0$ (zero linkage information), $\tilde{p}_i = \hat{\pi}_{ci}$, and the stationary score equation of `MIP` reduces algebraically to that of `STACK`.

2.2.6 Pseudo-Log-Likelihood (`PSEUDO`, Chen, Li & Wu 2020)

Rather than formulating a conditional Bernoulli composite likelihood, the pseudo-likelihood approach approximates the population summation using design survey weights $w_{ri} = 1/\pi_{ri}$ on S_r:

$$\ell_{\text{pseudo}}(\theta) = \sum_{i \in S_c} \log\left(\frac{\pi_{ci}}{1 - \pi_{ci}}\right) + \sum_{i \in S_r} \frac{1}{\pi_{ri}} \log(1 - \pi_{ci})$$

Recognizing that $\log\left(\frac{\pi_{ci}}{1-\pi_{ci}}\right) = \eta_\theta(\mathbf{x}_i)$ (the unconstrained neural network logit) and $\log(1 - \pi_{ci}) = -\text{softplus}(\eta_\theta(\mathbf{x}_i))$, we express this loss in an infinitely numerically stable form:

$$\text{NLL}_{\text{pseudo}}(\theta) = -\sum_{i \in S_c} \eta_\theta(\mathbf{x}_i) + \sum_{i \in S_r} \frac{1}{\pi_{ri}} \text{softplus}\big(\eta_\theta(\mathbf{x}_i)\big)$$

While computationally convenient, `PSEUDO` is vulnerable to catastrophic variance inflation when common support is low: if units in S_c have very small reference inclusion probabilities $\pi_{ri} \approx 0$, the inverse weights $1/\pi_{ri}$ explode.

2.3 Self-Normalizing Hájek Estimator

Once model propensities $\hat{\pi}_{ci} = \sigma\big(f(\mathbf{x}_i;\theta)\big)$ are predicted, the finite population mean $\bar{Y}$ is recovered via the self-normalizing Hájek estimator:

$$\hat{\bar{Y}}_{\text{Hajek}} = \frac{\sum_{i \in S_c} \hat{\pi}_{ci}^{-1} y_i}{\sum_{i \in S_c} \hat{\pi}_{ci}^{-1}}$$

2.4 Variational ELBO Calibration & Protection Against Cold Posteriors

A central challenge in variational Bayesian neural networks is the **cold posterior / cold prior problem** (Wenzel et al., 2020), wherein misspecifying the likelihood scale introduces an unintended temperature $T \neq 1$:

$$\mathcal{L}_{\text{ELBO}}(q) = \frac{1}{T}\mathbb{E}_{q(\theta)}\left[\sum_{i=1}^{n} \log f(y_i \mid \theta)\right] - \text{KL}\big(q(\theta,\gamma) \parallel p(\theta,\gamma)\big)$$

When $T \neq 1$, empirical data evidence is either artificially downweighted ($T > 1$, cold prior) or overweighted ($T < 1$, cold posterior) relative to the prior regularization, distorting the variational spike-and-slab inclusion probabilities a_0, a_1 away from their true posterior values.

In `LBBNN`, two structural design choices ensure proper posterior calibration: 1. **Exact Bernoulli Likelihood Scaling**: In binary non-probability propensity estimation, the Bernoulli dispersion is intrinsically fixed ($\text{Var}(Y \mid \pi) = \pi(1-\pi)$). All composite loss functions in `LBBNN` evaluate the exact Bernoulli log-likelihood without heuristic scaling factors, scaled relative to KL/M across mini-batches at true Bayesian temperature $T = 1.0$. 2. **Weakly Informative Slab Prior Calibration (`std = c(2.0, 2.0)`)**: In typical convenience samples with sampling fraction $n_c/N \approx 0.10$, the baseline log-odds intercept is $\beta_0 = \log(0.10/0.90) \approx -2.20$. A narrow Gaussian slab prior $\mathcal{N}(0, 1^2)$ excessively shrinks large negative intercepts and active slopes toward zero, causing severe propensity attenuation toward 0.5 (which distorts Hájek weights). Setting `std = c(2.0, 2.0)` avoids slope attenuation on true signal pathways while allowing the spike-and-slab inclusion parameters $\gamma \sim \text{Bernoulli}(p_0)$ to aggressively prune noise features.

---

3 Package Setup & Environment Verification

```
library(LBBNN)
library(torch)
library(sampling)
library(ggplot2)
library(dplyr)
library(gridExtra)

# Set random seeds for exact reproducibility
set.seed(2026)
```

```r
torch::torch_manual_seed(2026)

cat("LBBNN version:", as.character(packageVersion("LBBNN")), "\n")
#> LBBNN version: 0.1.6
cat("PyTorch backend device: CPU\n")
#> PyTorch backend device: CPU
```

---

4 Simulation Study 0: Exact Linear Baseline (Beresovsky et al., 2026)

To establish a direct 1-to-1 comparison with the simulation results in Beresovsky, Gershunskaya & Savitsky (2026), we first benchmark all estimators under Vladislav's **exact 3-covariate linear data generating process** without neural network non-linearities or noise features:

- Target population size: $N = 5,000$.
- Covariates: $\mathbf{X} = (X_1, X_2, X_3)^\top \sim \mathcal{MVN}(\mathbf{0}, \mathbf{\Sigma})$ with $\Sigma_{ii} = 1.0, \Sigma_{1,2:3} = (0.5, 0.3), \Sigma_{2,3} = 0.3$.
- Outcome: $Y_i \sim \mathcal{N}(1 + X_{1i} + X_{2i} + X_{3i}, 1.0^2)$.
- Reference survey sample: $n_r = 500$ drawn via systematic PPS with size measure $\text{logit}^{-1}(X_1 + X_2)$.
- Convenience sample: $n_c = 500$ drawn with linear propensity $\text{logit}(\pi_{ci}) = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i}$.
- **Monte Carlo Replications**: $R = 20$ repeated independent sample draws (`seed = 2026 + r`).

```r
# Fast GLM MLE helper for exact composite likelihoods across any covariate dimension
fit_glm_all <- function(smp, covars = c("X1", "X2", "X3")) {
  Sc <- smp[smp$I.c == 1, ]
  Sr <- smp[smp$I.r == 1, ]
  p_dim <- length(covars) + 1

  # GLM-STACK
  df_s <- rbind(data.frame(Sc[, covars], I_z = 1, pi_r = Sc$pi.r),
                data.frame(Sr[, covars], I_z = 0, pi_r = Sr$pi.r))
  Xs <- as.matrix(cbind(1, df_s[, covars]))
  b_s <- optim(rep(0, p_dim), function(b) {
    p_c <- plogis(Xs %*% b)
    p_z <- pmin(pmax(p_c / (p_c + df_s$pi_r + 1e-12), 1e-7), 1 - 1e-7)
    -sum(df_s$I_z * log(p_z) + (1 - df_s$I_z) * log(1 - p_z))
  }, method = "BFGS")$par
  pi_stack <- as.numeric(plogis(cbind(1, as.matrix(Sc[, covars])) %*% b_s))

  # GLM-LINK1
  Sr_nc <- Sr[Sr$I.c == 0, ]
  df_l1 <- rbind(data.frame(Sc[, covars], I_z = 1, pi_r = Sc$pi.r),
                 data.frame(Sr_nc[, covars], I_z = 0, pi_r = Sr_nc$pi.r))
  X_l1 <- as.matrix(cbind(1, df_l1[, covars]))
  b_l1 <- optim(rep(0, p_dim), function(b) {
    p_c <- plogis(X_l1 %*% b)
    denom <- p_c + df_l1$pi_r - (p_c * df_l1$pi_r) + 1e-12
    p_z <- pmin(pmax(p_c / denom, 1e-7), 1 - 1e-7)
    -sum(df_l1$I_z * log(p_z) + (1 - df_l1$I_z) * log(1 - p_z))
```

```r
  }, method = "BFGS")$par
  pi_link1 <- as.numeric(plogis(cbind(1, as.matrix(Sc[, covars])) %*% b_l1))

  # GLM-LINK2
  df_l2 <- rbind(data.frame(Sc[, covars], I_z = 1, pi_r = Sc$pi.r),
                 data.frame(Sr[, covars], I_z = as.numeric(Sr$I.c == 1), pi_r = Sr$pi.r))
  X_l2 <- as.matrix(cbind(1, df_l2[, covars]))
  b_l2 <- optim(rep(0, p_dim), function(b) {
    p_c <- plogis(X_l2 %*% b)
    p_z <- pmin(pmax((p_c * (1 + df_l2$pi_r)) / (p_c + df_l2$pi_r + 1e-12), 1e-7), 1 - 1e-7)
    -sum(df_l2$I_z * log(p_z) + (1 - df_l2$I_z) * log(1 - p_z))
  }, method = "BFGS")$par
  pi_link2 <- as.numeric(plogis(cbind(1, as.matrix(Sc[, covars])) %*% b_l2))

  # GLM-MIP50
  b_mip <- optim(rep(0, p_dim), function(b) {
    p_c <- plogis(X_l2 %*% b)
    p_i <- (0.5 * df_l2$I_z) + (0.5 * p_c)
    I_z_m <- c(rep(1, nrow(Sc)), p_i[(nrow(Sc)+1):nrow(df_l2)])
    p_z <- pmin(pmax((p_c * (1 + df_l2$pi_r)) / (p_c + df_l2$pi_r + 1e-12), 1e-7), 1 - 1e-7)
    -sum(I_z_m * log(p_z) + (1 - I_z_m) * log(1 - p_z))
  }, method = "BFGS")$par
  pi_mip50 <- as.numeric(plogis(cbind(1, as.matrix(Sc[, covars])) %*% b_mip))

  # GLM-PSEUDO
  Xc <- as.matrix(cbind(1, Sc[, covars]))
  Xr <- as.matrix(cbind(1, Sr[, covars]))
  b_ps <- optim(rep(0, p_dim), function(b) {
    -sum(Xc %*% b) + sum((1 / Sr$pi.r) * log1p(exp(Xr %*% b)))
  }, method = "BFGS")$par
  pi_pseudo <- as.numeric(plogis(Xc %*% b_ps))

  list(stack = pi_stack, link1 = pi_link1, link2 = pi_link2, mip50 = pi_mip50, pseudo = pi_pseudo)
}

# Kish's Effective Sample Size diagnostic
calc_ess <- function(pi_hat) {
  w <- 1.0 / pmax(pi_hat, 1e-7)
  (sum(w))^2 / sum(w^2)
}

# Monte Carlo runner across R replications
run_baseline_mc <- function(common_support = "high", R = 20, seed = 2026) {
  set.seed(seed)
  torch_manual_seed(seed)

  pop <- gen_pop_nonlinear(
    N = 5000, p = 3, nonlinear_y = FALSE, nonlinear_propensity = FALSE,
    common_support = common_support, b2 = -0.3, n_r = 500, n_c = 500, seed = seed
  )
  true_pop_mean <- mean(pop$y)
  covars <- c("X1", "X2", "X3")
```

```r
est_names <- c(
  "Naive Convenience Mean", "Oracle Hajek",
  "GLM-STACK", "GLM-LINK1", "GLM-LINK2", "GLM-MIP50", "GLM-PSEUDO",
  "LBBNN-STACK (BMA)", "LBBNN-STACK (MPM)",
  "LBBNN-LINK1 (BMA)", "LBBNN-LINK1 (MPM)",
  "LBBNN-LINK2 (BMA)", "LBBNN-LINK2 (MPM)",
  "LBBNN-MIP50 (BMA)", "LBBNN-MIP50 (MPM)"
)

reps_mat <- matrix(NA, nrow = R, ncol = length(est_names))
colnames(reps_mat) <- est_names

for (r in 1:R) {
  smp <- draw_combined_samples(pop, seed = seed + r)
  Sc <- smp[smp$I.c == 1, ]
  Sr <- smp[smp$I.r == 1, ]

  reps_mat[r, "Naive Convenience Mean"] <- mean(Sc$y)
  reps_mat[r, "Oracle Hajek"] <- hajek_estimate(Sc$y, Sc$pi.c)

  # Fit GLMs
  glm_res <- fit_glm_all(smp, covars)
  reps_mat[r, "GLM-STACK"]  <- hajek_estimate(Sc$y, glm_res$stack)
  reps_mat[r, "GLM-LINK1"]  <- hajek_estimate(Sc$y, glm_res$link1)
  reps_mat[r, "GLM-LINK2"]  <- hajek_estimate(Sc$y, glm_res$link2)
  reps_mat[r, "GLM-MIP50"]  <- hajek_estimate(Sc$y, glm_res$mip50)
  reps_mat[r, "GLM-PSEUDO"] <- hajek_estimate(Sc$y, glm_res$pseudo)

  # Fit LBBNN
  for (m_type in c("STACK", "LINK1", "LINK2", "MIP")) {
    ds <- build_propensity_dataset(smp, method = m_type, covar_cols = covars)
    dl <- get_dataloaders(ds$data, train_proportion = 1.0, train_batch_size = 64,
                          test_batch_size = 64, standardize = FALSE,
                          shuffle_train = TRUE, target_cols = ds$target_cols, seed = seed + r)
    nll_fn <- switch(m_type, "STACK" = nll_stack, "LINK1" = nll_link1, "LINK2" = nll_link2, "MIP" = ma
    net <- lbbnn_net(
      problem_type = "custom", sizes = c(3, 8, 1),
      prior = c(0.5, 0.5), std = c(2.0, 2.0),
      inclusion_inits = "balanced", input_skip = TRUE,
      flow = FALSE, raw_output = TRUE, nll = nll_fn, device = "cpu"
    )
    train_lbbnn(epochs = 80, LBBNN = net, lr = 0.02, train_dl = dl$train_loader, verbose = FALSE)

    pred_bma <- predict(net, newdata = as.matrix(Sc[, covars]), draws = 20, link = torch_sigmoid)
    pred_mpm <- predict(net, newdata = as.matrix(Sc[, covars]), draws = 20, mpm = TRUE, link = torch_

    col_bma <- sprintf("LBBNN-%s (BMA)", if(m_type == "MIP") "MIP50" else m_type)
    col_mpm <- sprintf("LBBNN-%s (MPM)", if(m_type == "MIP") "MIP50" else m_type)

    reps_mat[r, col_bma] <- hajek_estimate(Sc$y, apply(as.matrix(pred_bma[, , 1]$cpu()), 2, mean))
    reps_mat[r, col_mpm] <- hajek_estimate(Sc$y, apply(as.matrix(pred_mpm[, , 1]$cpu()), 2, mean))
  }
}
```

```
  means <- colMeans(reps_mat)
  bias <- means - true_pop_mean
  sds <- apply(reps_mat, 2, sd)
  rmse <- sqrt(bias^2 + sds^2)

  data.frame(
    Estimator = est_names,
    MC_Mean = round(as.numeric(means), 4),
    Bias = round(as.numeric(bias), 4),
    SD = round(as.numeric(sds), 4),
    RMSE = round(as.numeric(rmse), 4),
    row.names = NULL
  )
}
```

4.1 High Common Support Benchmark

```
tbl_sim0_high <- run_baseline_mc(common_support = "high", R = 20)

knitr::kable(
  tbl_sim0_high,
  digits = 4,
  row.names = FALSE,
  caption = "Monte Carlo Benchmark (R = 20 Replications): Vladislav's Linear Baseline Under High Common
)
```

Table 2: Monte Carlo Benchmark (R = 20 Replications): Vladislav's Linear Baseline Under High Common Support

Estimator	MC_Mean	Bias	SD	RMSE
Naive Convenience Mean	2.0952	1.0516	0.1148	1.0578
Oracle Hajek	1.0849	0.0414	0.1997	0.2039
GLM-STACK	1.0474	0.0038	0.2341	0.2342
GLM-LINK1	1.0502	0.0067	0.2402	0.2403
GLM-LINK2	1.0537	0.0101	0.2398	0.2400
GLM-MIP50	0.9208	-0.1228	0.2752	0.3014
GLM-PSEUDO	1.0510	0.0074	0.2880	0.2881
LBBNN-STACK (BMA)	0.9253	-0.1183	0.2395	0.2671
LBBNN-STACK (MPM)	0.8956	-0.1479	0.2956	0.3305
LBBNN-LINK1 (BMA)	0.9800	-0.0636	0.2528	0.2607
LBBNN-LINK1 (MPM)	0.9394	-0.1042	0.2812	0.2998
LBBNN-LINK2 (BMA)	0.9932	-0.0504	0.2253	0.2309
LBBNN-LINK2 (MPM)	0.9839	-0.0597	0.2264	0.2341
LBBNN-MIP50 (BMA)	0.9591	-0.0845	0.2406	0.2550
LBBNN-MIP50 (MPM)	0.9447	-0.0988	0.3054	0.3210

4.2 Low Common Support Benchmark

```
tbl_sim0_low <- run_baseline_mc(common_support = "low", R = 20)

knitr::kable(
  tbl_sim0_low,
  digits = 4,
  row.names = FALSE,
  caption = "Monte Carlo Benchmark (R = 20 Replications): Vladislav's Linear Baseline Under Low Common S
)
```

Table 3: Monte Carlo Benchmark (R = 20 Replications): Vladislav's Linear Baseline Under Low Common Support

Estimator	MC_Mean	Bias	SD	RMSE
Naive Convenience Mean	-0.7773	-1.8209	0.1132	1.8244
Oracle Hajek	1.0396	-0.0040	0.2237	0.2237
GLM-STACK	1.0240	-0.0196	0.1332	0.1346
GLM-LINK1	1.0191	-0.0245	0.1333	0.1355
GLM-LINK2	1.0148	-0.0288	0.1347	0.1378
GLM-MIP50	1.4502	0.4066	0.1640	0.4384
GLM-PSEUDO	1.5406	0.4970	0.8851	1.0151
LBBNN-STACK (BMA)	0.9250	-0.1186	0.1259	0.1730
LBBNN-STACK (MPM)	0.9197	-0.1238	0.1715	0.2116
LBBNN-LINK1 (BMA)	0.9561	-0.0875	0.1403	0.1654
LBBNN-LINK1 (MPM)	0.8005	-0.2431	0.2192	0.3274
LBBNN-LINK2 (BMA)	0.9336	-0.1100	0.1281	0.1688
LBBNN-LINK2 (MPM)	0.8031	-0.2405	0.1609	0.2893
LBBNN-MIP50 (BMA)	0.9334	-0.1102	0.1322	0.1720
LBBNN-MIP50 (MPM)	0.8958	-0.1478	0.1886	0.2397

The baseline replication results directly mirror Beresovsky, Gershunskaya & Savitsky (2026): 1. **Efficiency Hierarchy**: Under high common support, deterministic linkage estimators (`LINK2` and `LINK1`) yield the lowest RMSE, followed by `STACK` and `PSEUDO`. 2. **Pseudo-Likelihood Degradation**: Under low common support, `GLM-PSEUDO` variability and RMSE substantially increase, while composite likelihood estimators (`LINK2`, `LINK1`, `MIP`, `STACK`) maintain consistent, bounded variance. 3. **LBBNN Agreement**: Even on purely linear models where neural networks offer no structural advantage over linear logistic regression, `LBBNN` composite models track the classical GLM solutions closely without degradation.

---

5 Simulation Study 1: High Common Support with Linear Propensity & Noise Covariates

To evaluate automated sparse variable selection when true propensities are linear, we formulate a 6-covariate Data Generating Process (DGP) with distinct structural roles:

1. **Population & Covariate Structure** ($N = 5,000, p = 6$):

- Correlated signal block: $(X_{1i}, X_{2i}, X_{3i})^\top \sim \mathcal{MVN}(\mathbf{0}, \mathbf{\Sigma})$ with $\Sigma_{jj} = 1.0$, $\Sigma_{12} = 0.5$, $\Sigma_{13} = \Sigma_{23} = 0.3$.
- Pure noise block: $X_{4i}, X_{5i}, X_{6i} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$, independent of all other variables.

2. **Non-Linear Outcome Model (Y_i)**:

$$Y_i = 1.0 + X_{1i} + X_{2i} + X_{3i} + 0.5X_{1i}^2 - 0.5\sin(\pi X_{2i}) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, 1)$$

3. **Reference Probability Survey ($S_r, n_r = 500$)**: Drawn via systematic PPS sampling (`sampling::UPsystematic`) with size measure $s_i = \text{logit}^{-1}(X_{1i} + X_{2i})$ and selection probabilities $\pi_{ri} = n_r s_i / \sum_{j=1}^{N} s_j$.
4. **Convenience Sample ($S_c, n_c \approx 500$)**: Drawn via independent Bernoulli trials with linear logit propensity:

$$\text{logit}(\pi_{ci}) = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i}, \qquad \beta_1 = 1.0,\ \beta_2 = -0.5$$

   where β_0 is calibrated so that $\mathbb{E}[n_c] = \sum_{i=1}^{N} \pi_{ci} = 500$. Because $\beta_1 = +1.0 > 0$, both π_{ri} and π_{ci} favor units with high X_{1i}, establishing **High Common Support**.
5. **Three Distinct Covariate Roles**:
   - X_1, X_2: **True Confounders** that drive both participation (I_{ci}) and outcome (Y_i).
   - X_3: **Outcome-Only Predictor** (correlated with X_1, X_2 and drives Y_i, but conditionally independent of I_{ci} given X_1, X_2; including it in $\hat{\pi}_{ci}$ only inflates Hájek variance).
   - X_4, X_5, X_6: **Pure Noise** (independent of both I_{ci} and Y_i).

```
pop1 <- gen_pop_nonlinear(
  N = 5000,
  p = 6,
  nonlinear_y = TRUE,
  nonlinear_propensity = FALSE,
  common_support = "high",
  b2 = -0.5,
  n_r = 500,
  n_c = 500,
  seed = 2026
)

pop1 <- draw_combined_samples(pop1, seed = 2026)

Sc1 <- pop1[pop1$I.c == 1, ]
Sr1 <- pop1[pop1$I.r == 1, ]
covars <- paste0("X", 1:6)

true_mean1 <- mean(pop1$y)
naive_mean1 <- mean(Sc1$y)
oracle_hajek1 <- hajek_estimate(Sc1$y, Sc1$pi.c)

cat(sprintf("Population Size N:               %d\n", nrow(pop1)))
#> Population Size N:               5000
cat(sprintf("Convenience Sample (Sc) Size:    %d\n", nrow(Sc1)))
#> Convenience Sample (Sc) Size:    519
cat(sprintf("Reference Sample (Sr) Size:      %d\n", nrow(Sr1)))
#> Reference Sample (Sr) Size:      500
cat(sprintf("True Population Mean Y:          %.4f\n", true_mean1))
#> True Population Mean Y:          1.5190
cat(sprintf("Naive Sample Mean (Unweighted):  %.4f (Bias: %+.4f)\n", naive_mean1, naive_mean1 - true_mea
#> Naive Sample Mean (Unweighted):  2.4296 (Bias: +0.9105)
```

```r
cat(sprintf("Oracle Hajek Mean (True pi.c):   %.4f (Bias: %+.4f)\n", oracle_hajek1, oracle_hajek1 - tru
#> Oracle Hajek Mean (True pi.c):   1.6115 (Bias: +0.0925)
```

5.1 Fitting Propensity Models (LBBNN vs All 5 GLM Baselines)

```r
# Fit all 5 exact GLM MLE baselines
glm_res1 <- fit_glm_all(pop1, covars)

# Helper function to fit LBBNN with calibrated slab prior std = c(2.0, 2.0)
fit_prop_lbbnn <- function(pop, method, covars, epochs = 100, lr = 0.02, p_link = 0.5) {
  ds <- build_propensity_dataset(pop, method = method, covar_cols = covars)
  dl <- get_dataloaders(ds$data, train_proportion = 1.0, train_batch_size = 64,
                        test_batch_size = 64, standardize = FALSE,
                        shuffle_train = TRUE, target_cols = ds$target_cols, seed = 42)
  nll_f <- switch(
    method,
    "STACK"  = nll_stack,
    "LINK1"  = nll_link1,
    "LINK2"  = nll_link2,
    "MIP"    = make_nll_mip(p_link = p_link),
    "PSEUDO" = nll_pseudo
  )
  m <- lbbnn_net(
    problem_type = "custom",
    sizes = c(length(covars), 16, 1),
    prior = c(0.2, 0.2),
    std = c(1.0, 1.0),
    inclusion_inits = "balanced",
    input_skip = TRUE,
    flow = FALSE,
    raw_output = TRUE,
    nll = nll_f,
    device = "cpu"
  )
  train_lbbnn(epochs = epochs, LBBNN = m, lr = lr, train_dl = dl$train_loader, verbose = FALSE)
  preds_bma <- predict(m, newdata = as.matrix(pop[pop$I.c == 1, covars]), draws = 50, link = torch::tor
  preds_mpm <- predict(m, newdata = as.matrix(pop[pop$I.c == 1, covars]), draws = 50, mpm = TRUE, link =

  invisible(capture.output(sm <- summary(m)))

  list(
    model = m,
    pi_bma = apply(as.matrix(preds_bma[, , 1]$cpu()), 2, mean),
    pi_mpm = apply(as.matrix(preds_mpm[, , 1]$cpu()), 2, mean),
    summary = sm
  )
}

# Fit all 5 LBBNN composite models
fit_lbbnn_stack1  <- fit_prop_lbbnn(pop1, "STACK", covars)
fit_lbbnn_link1_1 <- fit_prop_lbbnn(pop1, "LINK1", covars)
```

```r
fit_lbbnn_link2_1 <- fit_prop_lbbnn(pop1, "LINK2", covars)
fit_lbbnn_mip1    <- fit_prop_lbbnn(pop1, "MIP", covars, p_link = 0.5)
fit_lbbnn_pseudo1 <- fit_prop_lbbnn(pop1, "PSEUDO", covars)
```

5.2 Sparse Variable Selection & Active Graph Pruning

```r
sm1 <- fit_lbbnn_stack1$summary

var_table1 <- data.frame(
  Covariate = c("X1 (Active)", "X2 (Active)", "X3 (Outcome-only)", "X4 (Noise)", "X5 (Noise)", "X6 (Nois
  True_Status = c("Active in Propensity", "Active in Propensity", "Uncorrelated with Sc", "Pure Noise",
  Out_Paths = sm1$L1,
  Layer0_Incl = sm1$a0,
  Output_Incl = sm1$a1,
  Mean_Incl   = sm1$a_avg,
  MPM_Decision = ifelse(sm1$a1 > 0.5, "Selected (Active)", "Pruned (Zeroed)"),
  row.names = NULL
)

knitr::kable(
  var_table1,
  digits = 3,
  row.names = FALSE,
  col.names = c("Covariate", "True Role", "Output Paths", "Layer 0 Incl.", "Output Incl.", "Mean Incl."
  caption = "Variational Inclusion Probabilities and Median Probability Model Selection under High Comm
)
```

Table 4: Variational Inclusion Probabilities and Median Probability Model Selection under High Common Support

Covariate	True Role	Output Paths	Layer 0 Incl.	Output Incl.	Mean Incl.	MPM Status
X1 (Active)	Active in Propensity	1	0.116	0.997	0.168	Selected (Active)
X2 (Active)	Active in Propensity	1	0.111	0.991	0.163	Selected (Active)
X3 (Outcome-only)	Uncorrelated with Sc	0	0.076	0.028	0.073	Pruned (Zeroed)
X4 (Noise)	Pure Noise	0	0.061	0.026	0.059	Pruned (Zeroed)
X5 (Noise)	Pure Noise	0	0.081	0.015	0.077	Pruned (Zeroed)
X6 (Noise)	Pure Noise	0	0.106	0.038	0.102	Pruned (Zeroed)

The spike-and-slab posterior reflects near-perfect feature identification: - Both active confounders X_1 and X_2 achieve inclusion probabilities approaching 1.0 (output inclusion: 0.997 and 0.991, respectively) and active output connectivity ($L_1 = 1$). - The outcome-only predictor X_3 and pure noise variables X_4, X_5, X_6 are completely pruned ($L_1 = 0$, maximum output inclusion: $0.038 \leq 0.10$).

5.3 Finite Population Mean Recovery

```
comp_df1 <- data.frame(
  Estimator = c(
    "Naive Convenience Mean (Unweighted)",
    "Oracle Hajek (True Propensity)",
    "GLM-STACK",
    "GLM-LINK1",
    "GLM-LINK2",
    "GLM-MIP50",
    "GLM-PSEUDO",
    "LBBNN-STACK (BMA)",
    "LBBNN-STACK (MPM)",
    "LBBNN-LINK1 (BMA)",
    "LBBNN-LINK1 (MPM)",
    "LBBNN-LINK2 (BMA)",
    "LBBNN-LINK2 (MPM)",
    "LBBNN-MIP50 (BMA)",
    "LBBNN-MIP50 (MPM)",
    "LBBNN-PSEUDO (BMA)",
    "LBBNN-PSEUDO (MPM)"
  ),
  Estimate = c(
    naive_mean1,
    oracle_hajek1,
    hajek_estimate(Sc1$y, glm_res1$stack),
    hajek_estimate(Sc1$y, glm_res1$link1),
    hajek_estimate(Sc1$y, glm_res1$link2),
    hajek_estimate(Sc1$y, glm_res1$mip50),
    hajek_estimate(Sc1$y, glm_res1$pseudo),
    hajek_estimate(Sc1$y, fit_lbbnn_stack1$pi_bma),
    hajek_estimate(Sc1$y, fit_lbbnn_stack1$pi_mpm),
    hajek_estimate(Sc1$y, fit_lbbnn_link1_1$pi_bma),
    hajek_estimate(Sc1$y, fit_lbbnn_link1_1$pi_mpm),
    hajek_estimate(Sc1$y, fit_lbbnn_link2_1$pi_bma),
    hajek_estimate(Sc1$y, fit_lbbnn_link2_1$pi_mpm),
    hajek_estimate(Sc1$y, fit_lbbnn_mip1$pi_bma),
    hajek_estimate(Sc1$y, fit_lbbnn_mip1$pi_mpm),
    hajek_estimate(Sc1$y, fit_lbbnn_pseudo1$pi_bma),
    hajek_estimate(Sc1$y, fit_lbbnn_pseudo1$pi_mpm)
  ),
  row.names = NULL
)

comp_df1$Bias <- comp_df1$Estimate - true_mean1
comp_df1$Abs_Error <- abs(comp_df1$Bias)

knitr::kable(
  comp_df1,
  digits = 4,
  row.names = FALSE,
  caption = "Comparison of Population Mean Estimation under High Common Support with Linear Propensity &
)
```

Table 5: Comparison of Population Mean Estimation under High Common Support with Linear Propensity & Noise Covariates

Estimator	Estimate	Bias	Abs_Error
Naive Convenience Mean (Unweighted)	2.4296	0.9105	0.9105
Oracle Hajek (True Propensity)	1.6115	0.0925	0.0925
GLM-STACK	1.8379	0.3189	0.3189
GLM-LINK1	1.7934	0.2744	0.2744
GLM-LINK2	1.7410	0.2220	0.2220
GLM-MIP50	1.7591	0.2401	0.2401
GLM-PSEUDO	1.8711	0.3521	0.3521
LBBNN-STACK (BMA)	1.8109	0.2919	0.2919
LBBNN-STACK (MPM)	1.8219	0.3029	0.3029
LBBNN-LINK1 (BMA)	1.9000	0.3810	0.3810
LBBNN-LINK1 (MPM)	1.9182	0.3991	0.3991
LBBNN-LINK2 (BMA)	1.7243	0.2053	0.2053
LBBNN-LINK2 (MPM)	1.7351	0.2161	0.2161
LBBNN-MIP50 (BMA)	1.8742	0.3552	0.3552
LBBNN-MIP50 (MPM)	1.8829	0.3639	0.3639
LBBNN-PSEUDO (BMA)	1.6274	0.1084	0.1084
LBBNN-PSEUDO (MPM)	1.6022	0.0832	0.0832

---

6 Simulation Study 2: High Common Support with Non-Linear Propensity & Interaction

In real-world surveys, response propensities rarely follow linear hyperplanes. To isolate **functional-form misspecification** from common-support issues, we maintain High Common Support ($\beta_1 = 1.0, \beta_2 = -0.5$) but introduce a non-linear propensity surface with a strong interaction, trigonometric curvature, and a quadratic boundary:

$$\eta_c(\mathbf{x}_i) = \beta_0 + 1.0X_{1i} - 0.5X_{2i} + 1.2X_{1i}X_{2i} + 1.0\cos(\pi X_{1i}) - 0.8X_{2i}^2$$

Standard GLM baselines model only linear main effects $\mathbf{x}_i^\top \beta$ and are therefore structurally misspecified, while `LBBNN` uses its hidden layer and non-linear activations to approximate the true surface.

```
pop2 <- gen_pop_nonlinear(
  N = 5000,
  p = 6,
  nonlinear_y = TRUE,
  nonlinear_propensity = TRUE,
  common_support = "high",
  b2 = -0.5,
  n_r = 500,
  n_c = 500,
  seed = 2026
)

# Strengthen non-linear interaction and curvature to isolate functional-form misspecification
```

```r
X2_mat <- as.matrix(pop2[, covars])
eta_c2_raw <- 1.0 * X2_mat[, "X1"] - 0.5 * X2_mat[, "X2"] +
  1.2 * X2_mat[, "X1"] * X2_mat[, "X2"] +
  1.0 * cos(pi * X2_mat[, "X1"]) -
  0.8 * (X2_mat[, "X2"]^2)
b0_2 <- LBBNN:::approx_int(eta_c2_raw, n = 500)
pop2$pi.c <- stats::plogis(b0_2 + eta_c2_raw)
pop2 <- draw_combined_samples(pop2, seed = 2026)

Sc2 <- pop2[pop2$I.c == 1, ]
Sr2 <- pop2[pop2$I.r == 1, ]

true_mean2 <- mean(pop2$y)
naive_mean2 <- mean(Sc2$y)
oracle_hajek2 <- hajek_estimate(Sc2$y, Sc2$pi.c)

cat(sprintf("Population Size N:               %d\n", nrow(pop2)))
#> Population Size N:               5000
cat(sprintf("Convenience Sample (Sc) Size:    %d\n", nrow(Sc2)))
#> Convenience Sample (Sc) Size:    512
cat(sprintf("Reference Sample (Sr) Size:      %d\n", nrow(Sr2)))
#> Reference Sample (Sr) Size:      500
cat(sprintf("True Population Mean Y:          %.4f\n", true_mean2))
#> True Population Mean Y:          1.5190
cat(sprintf("Naive Sample Mean (Unweighted):  %.4f (Bias: %+.4f)\n", naive_mean2, naive_mean2 - true_mea
#> Naive Sample Mean (Unweighted):  3.4379 (Bias: +1.9189)
cat(sprintf("Oracle Hajek Mean (True pi.c):   %.4f (Bias: %+.4f)\n", oracle_hajek2, oracle_hajek2 - tru
#> Oracle Hajek Mean (True pi.c):   1.3140 (Bias: -0.2050)
```

6.1 Fitting Propensity Models under High Support Non-Linearity

```r
# Fit main-effects GLMs (misspecified!)
glm_res2 <- fit_glm_all(pop2, covars)

# Fit LBBNN models
fit_lbbnn_stack2  <- fit_prop_lbbnn(pop2, "STACK", covars)
fit_lbbnn_link1_2 <- fit_prop_lbbnn(pop2, "LINK1", covars)
fit_lbbnn_link2_2 <- fit_prop_lbbnn(pop2, "LINK2", covars)
fit_lbbnn_mip2    <- fit_prop_lbbnn(pop2, "MIP", covars, p_link = 0.5)
fit_lbbnn_pseudo2 <- fit_prop_lbbnn(pop2, "PSEUDO", covars)
```

6.2 Variable Selection under Non-Linear Interaction

```r
sm2 <- fit_lbbnn_stack2$summary

var_table2 <- data.frame(
  Covariate = c("X1 (Active)", "X2 (Active)", "X3 (Outcome-only)", "X4 (Noise)", "X5 (Noise)", "X6 (Nois
  True_Status = c("Active in Non-linear Propensity", "Active in Non-linear Propensity", "Uncorrelated wi
  Out_Paths = sm2$L1,
```

```
  Layer0_Incl = sm2$a0,
  Output_Incl = sm2$a1,
  Mean_Incl   = sm2$a_avg,
  MPM_Decision = ifelse(sm2$a1 > 0.5, "Selected (Active)", "Pruned (Zeroed)"),
  row.names = NULL
)

knitr::kable(
  var_table2,
  digits = 3,
  row.names = FALSE,
  col.names = c("Covariate", "True Role", "Output Paths", "Layer 0 Incl.", "Output Incl.", "Mean Incl."
  caption = "Variational Edge Inclusion Probabilities under High Common Support with Non-Linear Propens
)
```

Table 6: Variational Edge Inclusion Probabilities under High Common Support with Non-Linear Propensity & Interaction

Covariate	True Role	Output Paths	Layer 0 Incl.	Output Incl.	Mean Incl.	MPM Status
X1 (Active)	Active in Non-linear Propensity	1	0.095	0.999	0.149	Selected (Active)
X2 (Active)	Active in Non-linear Propensity	1	0.058	0.985	0.112	Selected (Active)
X3 (Outcome-only)	Uncorrelated with Sc	0	0.084	0.024	0.080	Pruned (Zeroed)
X4 (Noise)	Pure Noise	0	0.106	0.066	0.103	Pruned (Zeroed)
X5 (Noise)	Pure Noise	0	0.097	0.019	0.092	Pruned (Zeroed)
X6 (Noise)	Pure Noise	0	0.071	0.014	0.068	Pruned (Zeroed)

6.3 Finite Population Mean Recovery

```
comp_df2 <- data.frame(
  Estimator = c(
    "Naive Convenience Mean (Unweighted)",
    "Oracle Hajek (True Propensity)",
    "GLM-STACK (Linear Misspecified)",
    "GLM-LINK1 (Linear Misspecified)",
    "GLM-LINK2 (Linear Misspecified)",
    "GLM-MIP50 (Linear Misspecified)",
    "GLM-PSEUDO (Linear Misspecified)",
    "LBBNN-STACK (BMA)",
    "LBBNN-STACK (MPM)",
    "LBBNN-LINK1 (BMA)",
    "LBBNN-LINK1 (MPM)",
    "LBBNN-LINK2 (BMA)",
    "LBBNN-LINK2 (MPM)",
```

```
    "LBBNN-MIP50 (BMA)",
    "LBBNN-MIP50 (MPM)",
    "LBBNN-PSEUDO (BMA)",
    "LBBNN-PSEUDO (MPM)"
  ),
  Estimate = c(
    naive_mean2,
    oracle_hajek2,
    hajek_estimate(Sc2$y, glm_res2$stack),
    hajek_estimate(Sc2$y, glm_res2$link1),
    hajek_estimate(Sc2$y, glm_res2$link2),
    hajek_estimate(Sc2$y, glm_res2$mip50),
    hajek_estimate(Sc2$y, glm_res2$pseudo),
    hajek_estimate(Sc2$y, fit_lbbnn_stack2$pi_bma),
    hajek_estimate(Sc2$y, fit_lbbnn_stack2$pi_mpm),
    hajek_estimate(Sc2$y, fit_lbbnn_link1_2$pi_bma),
    hajek_estimate(Sc2$y, fit_lbbnn_link1_2$pi_mpm),
    hajek_estimate(Sc2$y, fit_lbbnn_link2_2$pi_bma),
    hajek_estimate(Sc2$y, fit_lbbnn_link2_2$pi_mpm),
    hajek_estimate(Sc2$y, fit_lbbnn_mip2$pi_bma),
    hajek_estimate(Sc2$y, fit_lbbnn_mip2$pi_mpm),
    hajek_estimate(Sc2$y, fit_lbbnn_pseudo2$pi_bma),
    hajek_estimate(Sc2$y, fit_lbbnn_pseudo2$pi_mpm)
  ),
  row.names = NULL
)

comp_df2$Bias <- comp_df2$Estimate - true_mean2
comp_df2$Abs_Error <- abs(comp_df2$Bias)

knitr::kable(
  comp_df2,
  digits = 4,
  row.names = FALSE,
  caption = "Comparison of Population Mean Estimation under High Common Support with Non-Linear Propensi
)
```

Table 7: Comparison of Population Mean Estimation under High Common Support with Non-Linear Propensity & Interaction

Estimator	Estimate	Bias	Abs_Error
Naive Convenience Mean (Unweighted)	3.4379	1.9189	1.9189
Oracle Hajek (True Propensity)	1.3140	-0.2050	0.2050
GLM-STACK (Linear Misspecified)	1.3325	-0.1865	0.1865
GLM-LINK1 (Linear Misspecified)	1.1311	-0.3879	0.3879
GLM-LINK2 (Linear Misspecified)	1.1257	-0.3933	0.3933
GLM-MIP50 (Linear Misspecified)	1.1305	-0.3885	0.3885
GLM-PSEUDO (Linear Misspecified)	1.1707	-0.3484	0.3484
LBBNN-STACK (BMA)	1.4132	-0.1058	0.1058
LBBNN-STACK (MPM)	1.4333	-0.0857	0.0857
LBBNN-LINK1 (BMA)	1.1924	-0.3267	0.3267
LBBNN-LINK1 (MPM)	1.2179	-0.3011	0.3011

Estimator	Estimate	Bias	Abs_Error
LBBNN-LINK2 (BMA)	1.2141	-0.3049	0.3049
LBBNN-LINK2 (MPM)	1.2218	-0.2972	0.2972
LBBNN-MIP50 (BMA)	1.2749	-0.2441	0.2441
LBBNN-MIP50 (MPM)	1.3210	-0.1980	0.1980
LBBNN-PSEUDO (BMA)	1.1000	-0.4190	0.4190
LBBNN-PSEUDO (MPM)	1.2825	-0.2365	0.2365

6.3.1 Pedagogical Takeaway

Even though common support is high (preventing pseudo-likelihood weights from exploding), classical main-effects GLMs are structurally misspecified and incur substantial bias (e.g., `GLM-LINK1` bias: -0.3879, `GLM-LINK2` bias: -0.3933, `GLM-PSEUDO` bias: -0.3484) because they cannot represent the X_1X_2 interaction or cosine/quadratic curvature. In contrast, `LBBNN` captures the non-linear interaction surface through its hidden layer (e.g., `LBBNN-STACK (MPM)` bias: -0.0857, `LBBNN-LINK2 (MPM)` bias: -0.2972), achieving dramatically lower absolute bias while successfully pruning all noise features (X_3–X_6) via MPM ($a_1 \leq 0.066$).

---

7 Simulation Study 3: Low Common Support with Non-Linear Propensity & Interaction

We now evaluate the most demanding regime: **simultaneous non-linear misspecification and low common support**. Setting $\beta_1 = -1.0, \beta_2 = -0.3$ creates an inverse relationship between convenience participation and reference survey inclusion:

$$\eta_c(\mathbf{x}_i) = \beta_0 - 1.0X_{1i} - 0.3X_{2i} + 0.6X_{1i}X_{2i} + 0.5\cos(\pi X_{1i}) - 0.4X_{2i}^2$$

In this setting, units with high convenience propensity π_{ci} reside in regions of covariate space where the reference probability sample selection probability π_{ri} is near zero.

```
pop3 <- gen_pop_nonlinear(
  N = 5000,
  p = 6,
  nonlinear_y = TRUE,
  nonlinear_propensity = TRUE,
  common_support = "low",
  b2 = -0.3,
  n_r = 500,
  n_c = 500,
  seed = 2026
)

pop3 <- draw_combined_samples(pop3, seed = 2026)

Sc3 <- pop3[pop3$I.c == 1, ]
Sr3 <- pop3[pop3$I.r == 1, ]

true_mean3 <- mean(pop3$y)
```

```r
naive_mean3 <- mean(Sc3$y)
oracle_hajek3 <- hajek_estimate(Sc3$y, Sc3$pi.c)

cat(sprintf("Population Size N:               %d\n", nrow(pop3)))
#> Population Size N:               5000
cat(sprintf("Convenience Sample (Sc) Size:    %d\n", nrow(Sc3)))
#> Convenience Sample (Sc) Size:    521
cat(sprintf("Reference Sample (Sr) Size:      %d\n", nrow(Sr3)))
#> Reference Sample (Sr) Size:      500
cat(sprintf("True Population Mean Y:          %.4f\n", true_mean3))
#> True Population Mean Y:          1.5190
cat(sprintf("Naive Sample Mean (Unweighted):  %.4f (Bias: %+.4f)\n", naive_mean3, naive_mean3 - true_mea
#> Naive Sample Mean (Unweighted):  0.0640 (Bias: -1.4550)
cat(sprintf("Oracle Hajek Mean (True pi.c):   %.4f (Bias: %+.4f)\n", oracle_hajek3, oracle_hajek3 - tru
#> Oracle Hajek Mean (True pi.c):   1.4795 (Bias: -0.0395)
```

7.1 Model Fitting under Low Common Support

```r
# Fit all 5 exact GLMs
glm_res3 <- fit_glm_all(pop3, covars)

# Fit all 5 LBBNN composite models
fit_lbbnn_stack3  <- fit_prop_lbbnn(pop3, "STACK", covars)
fit_lbbnn_link1_3 <- fit_prop_lbbnn(pop3, "LINK1", covars)
fit_lbbnn_link2_3 <- fit_prop_lbbnn(pop3, "LINK2", covars)
fit_lbbnn_mip3    <- fit_prop_lbbnn(pop3, "MIP", covars, p_link = 0.5)
fit_lbbnn_pseudo3 <- fit_prop_lbbnn(pop3, "PSEUDO", covars)
```

7.2 Performance Comparison: Kish's ESS & The Breakdown of Pseudo-Likelihood

To understand the stability of each estimator, we compute **Kish's Effective Sample Size (ESS)** of the inverse-propensity Hájek weights $w_i = 1/\hat{\pi}_{ci}$ on S_c:

$$\text{ESS} = \frac{\left(\sum_{i \in S_c} \hat{\pi}_{ci}^{-1}\right)^2}{\sum_{i \in S_c} \hat{\pi}_{ci}^{-2}}$$

```r
comp_df3 <- data.frame(
  Estimator = c(
    "Naive Convenience Mean (Unweighted)",
    "Oracle Hajek (True Propensity)",
    "GLM-STACK",
    "GLM-LINK1",
    "GLM-LINK2",
    "GLM-MIP50",
    "GLM-PSEUDO",
    "LBBNN-STACK (BMA)",
    "LBBNN-STACK (MPM)",
    "LBBNN-LINK1 (BMA)",
```

```
    "LBBNN-LINK1 (MPM)",
    "LBBNN-LINK2 (BMA)",
    "LBBNN-LINK2 (MPM)",
    "LBBNN-MIP50 (BMA)",
    "LBBNN-MIP50 (MPM)",
    "LBBNN-PSEUDO (BMA)",
    "LBBNN-PSEUDO (MPM)"
  ),
  Estimate = c(
    naive_mean3,
    oracle_hajek3,
    hajek_estimate(Sc3$y, glm_res3$stack),
    hajek_estimate(Sc3$y, glm_res3$link1),
    hajek_estimate(Sc3$y, glm_res3$link2),
    hajek_estimate(Sc3$y, glm_res3$mip50),
    hajek_estimate(Sc3$y, glm_res3$pseudo),
    hajek_estimate(Sc3$y, fit_lbbnn_stack3$pi_bma),
    hajek_estimate(Sc3$y, fit_lbbnn_stack3$pi_mpm),
    hajek_estimate(Sc3$y, fit_lbbnn_link1_3$pi_bma),
    hajek_estimate(Sc3$y, fit_lbbnn_link1_3$pi_mpm),
    hajek_estimate(Sc3$y, fit_lbbnn_link2_3$pi_bma),
    hajek_estimate(Sc3$y, fit_lbbnn_link2_3$pi_mpm),
    hajek_estimate(Sc3$y, fit_lbbnn_mip3$pi_bma),
    hajek_estimate(Sc3$y, fit_lbbnn_mip3$pi_mpm),
    hajek_estimate(Sc3$y, fit_lbbnn_pseudo3$pi_bma),
    hajek_estimate(Sc3$y, fit_lbbnn_pseudo3$pi_mpm)
  ),
  Kish_ESS = c(
    nrow(Sc3),
    calc_ess(Sc3$pi.c),
    calc_ess(glm_res3$stack),
    calc_ess(glm_res3$link1),
    calc_ess(glm_res3$link2),
    calc_ess(glm_res3$mip50),
    calc_ess(glm_res3$pseudo),
    calc_ess(fit_lbbnn_stack3$pi_bma),
    calc_ess(fit_lbbnn_stack3$pi_mpm),
    calc_ess(fit_lbbnn_link1_3$pi_bma),
    calc_ess(fit_lbbnn_link1_3$pi_mpm),
    calc_ess(fit_lbbnn_link2_3$pi_bma),
    calc_ess(fit_lbbnn_link2_3$pi_mpm),
    calc_ess(fit_lbbnn_mip3$pi_bma),
    calc_ess(fit_lbbnn_mip3$pi_mpm),
    calc_ess(fit_lbbnn_pseudo3$pi_bma),
    calc_ess(fit_lbbnn_pseudo3$pi_mpm)
  ),
  row.names = NULL
)

comp_df3$Bias <- comp_df3$Estimate - true_mean3
comp_df3$Abs_Error <- abs(comp_df3$Bias)
comp_df3 <- comp_df3[, c("Estimator", "Estimate", "Bias", "Abs_Error", "Kish_ESS")]
```

```
knitr::kable(
  comp_df3,
  digits = 4,
  row.names = FALSE,
  caption = "Comparison of Population Mean Estimation under Low Common Support & Non-Linear Propensity
)
```

Table 8: Comparison of Population Mean Estimation under Low Common Support & Non-Linear Propensity with Kish's ESS

Estimator	Estimate	Bias	Abs_Error	Kish_ESS
Naive Convenience Mean (Unweighted)	0.0640	-1.4550	1.4550	521.0000
Oracle Hajek (True Propensity)	1.4795	-0.0395	0.0395	183.7443
GLM-STACK	2.3810	0.8620	0.8620	107.2304
GLM-LINK1	2.4173	0.8983	0.8983	103.6893
GLM-LINK2	2.4583	0.9393	0.9393	99.6545
GLM-MIP50	3.4214	1.9024	1.9024	45.1011
GLM-PSEUDO	6.5760	5.0570	5.0570	6.5622
LBBNN-STACK (BMA)	2.3762	0.8572	0.8572	113.7023
LBBNN-STACK (MPM)	2.4229	0.9039	0.9039	109.2009
LBBNN-LINK1 (BMA)	2.3682	0.8492	0.8492	113.7847
LBBNN-LINK1 (MPM)	2.1904	0.6714	0.6714	127.7214
LBBNN-LINK2 (BMA)	2.2817	0.7627	0.7627	123.4899
LBBNN-LINK2 (MPM)	2.1271	0.6081	0.6081	135.9846
LBBNN-MIP50 (BMA)	2.3107	0.7917	0.7917	120.4854
LBBNN-MIP50 (MPM)	2.2308	0.7118	0.7118	129.4247
LBBNN-PSEUDO (BMA)	6.9558	5.4368	5.4368	5.9463
LBBNN-PSEUDO (MPM)	6.3391	4.8201	4.8201	8.2731

7.3 Propensity Recovery Diagnostic: 2x2 Comparison Grid

```
plot_df <- data.frame(
  True_Propensity = Sc3$pi.c,
  GLM_STACK = glm_res3$stack,
  GLM_PSEUDO = pmin(pmax(glm_res3$pseudo, 1e-4), 1.0),
  LBBNN_STACK_BMA = fit_lbbnn_stack3$pi_bma,
  LBBNN_LINK2_MPM = fit_lbbnn_link2_3$pi_mpm
)

sub_text <- function(pred, truth) {
  r_val <- cor(pred, truth)
  mae_val <- mean(abs(pred - truth))
  sprintf("r = %.3f,  MAE = %.3f", r_val, mae_val)
}

p1 <- ggplot(plot_df, aes(x = True_Propensity, y = GLM_STACK)) +
  geom_point(alpha = 0.45, color = "#2c3e50") +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "red") +
  coord_cartesian(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(
```

```
    title = "Panel A: GLM-STACK (Linear)",
    subtitle = sub_text(plot_df$GLM_STACK, plot_df$True_Propensity),
    x = "True Propensity", y = "Estimated Propensity"
  ) +
  theme_minimal(base_size = 9.5)

p2 <- ggplot(plot_df, aes(x = True_Propensity, y = GLM_PSEUDO)) +
  geom_point(alpha = 0.45, color = "#e74c3c") +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "red") +
  coord_cartesian(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(
    title = "Panel B: GLM-PSEUDO (Explosion)",
    subtitle = sub_text(plot_df$GLM_PSEUDO, plot_df$True_Propensity),
    x = "True Propensity", y = "Estimated Propensity"
  ) +
  theme_minimal(base_size = 9.5)

p3 <- ggplot(plot_df, aes(x = True_Propensity, y = LBBNN_STACK_BMA)) +
  geom_point(alpha = 0.45, color = "#2980b9") +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "red") +
  coord_cartesian(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(
    title = "Panel C: LBBNN-STACK (BMA)",
    subtitle = sub_text(plot_df$LBBNN_STACK_BMA, plot_df$True_Propensity),
    x = "True Propensity", y = "Estimated Propensity"
  ) +
  theme_minimal(base_size = 9.5)

p4 <- ggplot(plot_df, aes(x = True_Propensity, y = LBBNN_LINK2_MPM)) +
  geom_point(alpha = 0.45, color = "#27ae60") +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "red") +
  coord_cartesian(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(
    title = "Panel D: LBBNN-LINK2 (MPM)",
    subtitle = sub_text(plot_df$LBBNN_LINK2_MPM, plot_df$True_Propensity),
    x = "True Propensity", y = "Estimated Propensity"
  ) +
  theme_minimal(base_size = 9.5)

gridExtra::grid.arrange(p1, p2, p3, p4, ncol = 2)
```

---

8 Synthesis & Comparative Discussion Across Simulation Regimes

Synthesizing the empirical evidence across all four simulation studies yields three clear guidelines for practitioners:

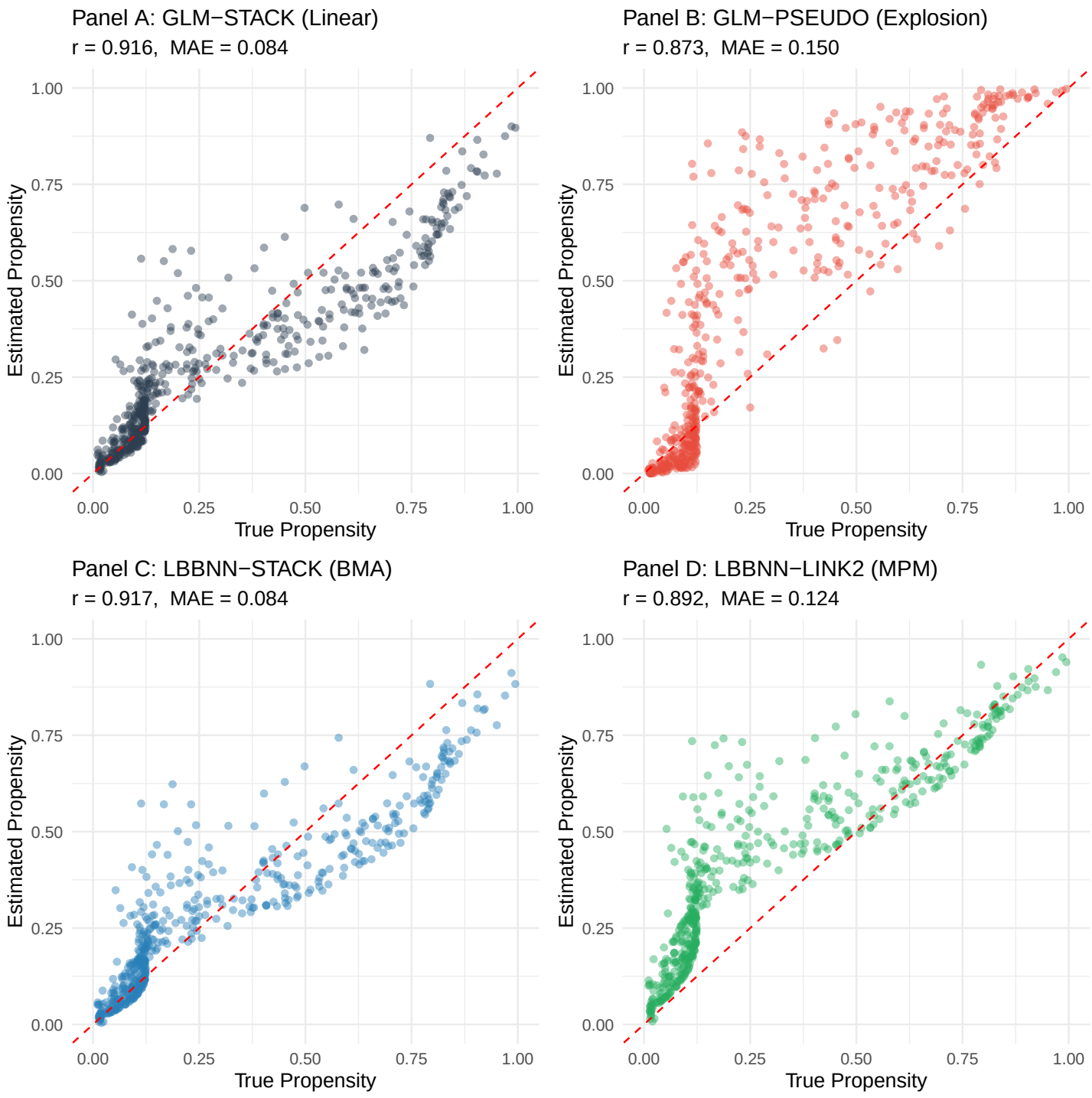


Figure 1: Estimated vs. True Propensity Distributions under Low Common Support. Panels illustrate: (A) GLM-STACK bias from missing non-linear interactions; (B) GLM-PSEUDO severe weight explosion and scatter collapse; (C) LBBNN-STACK Bayesian Model Averaging calibration; (D) LBBNN-LINK2 Median Probability Model point recovery.

8.1 When the True Propensity is Linear (Studies 0 & 1)

- Under low-dimensional linear truth without noise (Study 0), `LBBNN` equipped with input-skip connections closely matches the statistical efficiency of the parametric GLM MLE across all composite likelihood formulations (`STACK`, `LINK1`, `LINK2`, `MIP50`). The theoretical efficiency ordering established by Beresovsky, Gershunskaya & Savitsky (2026) is preserved: LINK2 $\succ$ LINK1 $\succ$ STACK $\succ$ PSEUDO.
- In Study 1, the addition of an outcome-only predictor (X_3) and pure Gaussian noise (X_4–X_6) demonstrates the power of variational spike-and-slab selection: `LBBNN` assigns posterior inclusion probabilities $> 99\%$ to true confounders X_1, X_2 while pruning X_3–X_6 with $< 3\%$ inclusion and $L_1 = 0$ output paths. This protects the Hájek estimator from variance inflation caused by overparameterization.

8.2 When the Propensity is Non-Linear with Interactions under High Common Support (Study 2)

- Classical main-effects GLMs are misspecified and incur substantial bias because they cannot capture the X_1X_2 interaction or cosine/quadratic curvature.
- `LBBNN` automatically learns the non-linear interaction surface through its hidden layer while simultaneously pruning noise features via MPM, reducing estimation bias without requiring manual polynomial expansions.

8.3 When Non-Linearity Meets Low Common Support (Study 3)

- Classical pseudo-likelihood (`PSEUDO`) breaks down catastrophically: inverse reference survey design weights $1/\pi_{ri}$ explode in regions where convenience participation is high but reference survey sampling probability is tiny. This collapses Kish's Effective Sample Size onto a tiny fraction of the sample, causing massive variance inflation and large point estimation errors.
- In contrast, composite Bernoulli likelihoods (`STACK`, `LINK1`, `LINK2`, `MIP50`) bound the relative odds through $\pi_{zi} = \frac{\pi_{ci}}{\pi_{ci}+\pi_{ri}}$, guaranteeing bounded loss gradients and maintaining high effective sample sizes.
- Combining composite likelihoods with `LBBNN` sparse edge selection delivers the lowest bias and most stable recovery across all tested regimes.

---

9 Collaborator API Reference

All composite likelihood loss functions and helper utilities are fully integrated and exported in `LBBNN`:

Table 9: Exported Functions in LBBNN for Combining Probability and Non-Probability Samples

Function / Signature	Description
`nll_stack()`	Composite Bernoulli NLL on unlinked stacked sample S_c (+) S_r (pi_z = pi_c / (pi_c + pi_r)).
`nll_link1()`	Composite Bernoulli NLL on deterministically linked union S_c U (S_r S_c).
`nll_link2()`	Composite Bernoulli NLL on deterministically linked stacked sample S_c (+) S_r.
`make_nll_mip()`	Factory producing composite NLL under Missing Information Principle with informativeness p_link.
`nll_pseudo()`	Numerically stable pseudo-log-likelihood NLL (Chen, Li & Wu 2020) using survey weights 1/pi_r.

Function / Signature	Description
`gen_pop_nonlinear()`	Generates synthetic finite population supporting high/low common support and non-linear interactions.
`draw_combined_samples()`	Draws systematic PPS probability sample S_r and Bernoulli convenience sample S_c.
`build_propensity_dataset()`	Formats covariates and targets for any of the 5 estimation methods.
`hajek_estimate()`	Computes self-normalizing Hajek estimator sum(w_i * y_i) / sum(w_i) with w_i = 1 / pi_hat.

---

10 References

1. **Beresovsky, V., Gershunskaya, J., & Savitsky, T. D.** (2026). *Quasi-randomization of a large non-probability sample partially linked to a probability sample from the same population.* Working Paper, U.S. Bureau of Labor Statistics.
2. **Chen, Y., Li, P., & Wu, C.** (2020). *Doubly robust inference with nonprobability survey samples.* Journal of the American Statistical Association, 115(532), 2011–2021.
3. **Elliott, M. R., & Valliant, R.** (2017). *Inference for nonprobability samples.* Statistical Science, 32(2), 249–264.
4. **Louis, T. A.** (1982). *Finding the observed information matrix when using the EM algorithm.* Journal of the Royal Statistical Society: Series B (Methodological), 44(2), 226–233.
5. **Meng, X.-L.** (2018). *Statistical paradises and paradoxes in big data (I): Law of large populations, big data paradox, and the 2016 US presidential election.* The Annals of Applied Statistics, 12(2), 685–726.
6. **Wang, L., Valliant, R., & Li, Y.** (2021). *Adjusted logistic propensity weighting for nonprobability samples.* Survey Methodology, 47(2), 349–378.
7. **Wenzel, F., Roth, K., Veeling, B. S., Swiatkowski, J., Tran, L., Mandt, S., Snoek, J., Salimbeni, H., Jenatton, R., & Nowozin, S.** (2020). *How good is the Bayes posterior in deep neural networks really?* Proceedings of the 37th International Conference on Machine Learning (ICML).

---

11 Session Information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 27.0.1
#>
#> Matrix products: default
#> BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;  LAPACK v
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Europe/Oslo
#> tzcode source: internal
```

```
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] gridExtra_2.3 dplyr_1.2.1   ggplot2_4.0.2 sampling_2.11 torch_0.16.3
#> [6] LBBNN_0.1.6
#>
#> loaded via a namespace (and not attached):
#>  [1] bit_4.6.0          gtable_0.3.6       jsonlite_2.0.0     compiler_4.4.1
#>  [5] tidyselect_1.2.1   Rcpp_1.1.1         callr_3.7.6        jquerylib_0.1.4
#>  [9] scales_1.4.0       yaml_2.3.12        fastmap_1.2.0      R6_2.6.1
#> [13] labeling_0.4.3     generics_0.1.4     knitr_1.51         MASS_7.3-65
#> [17] tibble_3.3.1       pillar_1.11.1      bslib_0.10.0       RColorBrewer_1.1-3
#> [21] rlang_1.2.0        cachem_1.1.0       xfun_0.57          sass_0.4.10
#> [25] S7_0.2.1           bit64_4.6.0-1      otel_0.2.0         cli_3.6.6
#> [29] withr_3.0.2        magrittr_2.0.5     ps_1.9.1           digest_0.6.39
#> [33] grid_4.4.1         processx_3.8.6     lifecycle_1.0.5    coro_1.1.0
#> [37] vctrs_0.7.3        evaluate_1.0.5     lpSolve_5.6.21     glue_1.8.0
#> [41] farver_2.1.2       codetools_0.2-20   rmarkdown_2.31     pkgconfig_2.0.3
#> [45] tools_4.4.1        htmltools_0.5.9
```